CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 14

Admin Dashboard with Role-Based Access, Live Statistics and Audit Logging

PHP 8 · MySQLi prepared statements · MariaDB · Bootstrap 5 · role-based access control · audit log

Report – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

New report. No Word or PDF report existed for Practical 14: the repository only contained `Practical-14/README.md`, which describes a standalone dashboard on the `user_registration` database of Practicals 9 and 10. This report describes Practical 14 as it is implemented in the canonical integrated portal `studenthub/` (commit `05927b3`, database `studenthub_db`): the Admin Console overview with live statistics, role-based navigation from the `menu_items` table, account status (disable / enable), one consistent HTTP 403 page and the filtered audit log. All screenshots were taken from `studenthub/` with Microsoft Edge (headless, Playwright) on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Problem Definition and Objectives

Topic (from `Practical-14/README.md`): *Admin Dashboard with Role-Based Access, Statistics & Audit Logging*. The practical status table of the root `README.md` lists it as "Admin dashboard, audit logging, 403".

The first version was a standalone folder `Practical-14/` (Bootstrap 5 + PHP/MySQLi on the `user_registration` database; compared with the integrated version in Section 3). Its navigation hub was not filtered by role, its `schema.sql` inserted six sample audit rows (four of their actions are never written by the code) and re-importing it erased the recorded audit log; no screenshots or report were made for it.

The task of the integration was to give the one portal `studenthub/` – which already had the logins, student records and events of Practicals 8–13 – a real administrator dashboard on its own database. Objectives:

- Show live statistics of `studenthub_db`, counted on every page load: students (active / inactive), events (upcoming / open), registrations (active / cancelled / attended), accounts (per role and disabled) and the audit log (entries and security blocks in the last 7 days).
- Show the latest event registrations (student and event), the newest accounts and the most recent activity.
- Build the navigation from `menu_items` with a role hierarchy student < editor < admin, so that an editor never sees an administrator link; hide links to pages that do not exist and escape every value read from the database.
- Give every request by a signed-in student or editor to any Admin Console tab or action the same HTTP 403 page and an audit entry.
- Let administrators disable and enable accounts: a disabled account cannot sign in and loses its open sessions; the own account and the last active administrator are protected.
- Offer the audit log with filters by action, status and user.
- Record automated HTTP and browser tests and genuine Microsoft Edge screenshots.

2. Tools and Technologies

Tool / technology	Use in this practical
PHP 8.2.12 (XAMPP), MySQLi	Admin Console, statistics, account status and audit queries – every query with user input is a prepared statement; mysqli runs in exception mode
MariaDB 10.4.32	The portal database `studenthub_db`; migration 004 adds `users.status`, the role check `chk_users_role` and the roles of `menu_items`
Bootstrap 5.3.2 and Bootstrap Icons 1.11.1 (CDN) + `assets/css/bootstrap-theme.css`	Layout of the Admin Console (cards, tabs, tables, badges)
Python 3 test scripts	`tests/test_dashboard.py` (HTTP against PHP's built-in server on Windows 11), `database/tests/test_004_admin_dashboard.py`

	(migration)
Playwright + Microsoft Edge (headless)	`tests/browser_dashboard.py` – browser checks and the screenshots

3. Source Files

File	Role in Practical 14
`studenthub/admin_dashboard.php`	Admin Console: five statistics cards, navigation box for the role, tabs Overview, Student Records, Campus Events, User Accounts, Inquiries, Security Audit Trail; POST handlers for disable / enable / delete of accounts
`studenthub/includes/dashboard.php`	`dashboard_stats()`, `latest_registrations()`, `newest_accounts()`, `active_admin_count()`, `find_account()`, `set_account_status()`, `audit_actions()`, `audit_filters()`, `audit_list()`
`studenthub/includes/navigation.php`	`ROLE_RANK`, `portal_page_exists()`, `menu_for_role()`
`studenthub/sidebar.php`	Grouped sidebar of every portal page; adds a *Shortcuts* group from the `menu_items` rows of the user's role that the fixed groups do not contain
`studenthub/includes/auth.php`	`STUDENTHUB_PERMISSIONS`, `require_permission()`, `account_block_reason()`, `refresh_user_from_database()`, `validate_session()`
`studenthub/login.php`	Refuses a disabled account after the password check (`LOGIN_DENIED`)
`studenthub/403.php`	The one shared “Access Denied” page (HTTP 403)
`studenthub/audit_logger.php`	`log_audit()` – prepared INSERT into `audit_logs`
`database/migrations/004_admin_dashboard.sql` / `.down.sql`	Migration 004 and its rollback
`database/tests/test_004_admin_dashboard.py`	Tests of migration 004 + read-only live check

The standalone `Practical-14/` folder is kept unchanged as historical evidence. The table compares it with the integrated version:

Aspect	Standalone `Practical-14/` (README)	Integrated `studenthub/`
Database	`user_registration` (shared with Practicals 9 and 10)	`studenthub_db`, the one database of the portal
Statistics	Row counts of `users`, `students`, `events`, `audit_logs`	Five counters with breakdowns, counted on every page load
Recent data	Five most recent accounts	Latest event registrations, newest accounts, recent activity
Navigation	Every `menu_items` row shown to the admin	Rows up to the own role; missing pages hidden; values escaped; sidebar Shortcuts
Access control	Only `role = admin` may open `dashboard.php`	Permission checked for the console and for every action; same 403 page and `ACCESS_DENIED` entry
Account status	–	`users.status` active / disabled; own account and last active admin protected
Audit log	Six newest rows, plus sample rows from `schema.sql`	Newest 100 matching entries with action / status / user filters; only real entries

4. Database

Practical 14 uses the existing tables of `studenthub_db` and changes them only through migration 004 (`database/migrations/004_admin_dashboard.sql`). Its preconditions (migration 003 recorded, every account has the role `admin`, `editor` or `student`) run first, so nothing is altered if one fails. It then converts the menu role `user` (not a portal role) to `student`, points links to old standalone pages or missing anchors at the integrated pages, adds `users.status` (every existing account stays `active`) and the role check, verifies the result and records the row `004` in `schema_migrations`. The row linking to the non-existent `faculty.php` is kept – the portal hides links to missing pages. The script is idempotent; the rollback removes `users.status` and the check and maps `student` back to `user`.

database/migrations/004_admin_dashboard.sql (lines 40–55) – the schema changes of migration 004

```
40  -- 1. menu_items.min_role: 'user' -> 'student' (widen, convert, narrow)
41  ALTER TABLE menu_items MODIFY min_role ENUM('user', 'student', 'editor', 'admin') NOT NULL DEFAULT
    'student';
42  UPDATE menu_items SET min_role = 'student' WHERE min_role = 'user';
43  ALTER TABLE menu_items MODIFY min_role ENUM('student', 'editor', 'admin') NOT NULL DEFAULT 'student';
44
45  -- 2. links to old standalone pages or missing anchors -> the integrated pages
46  UPDATE menu_items SET link = 'profile.php'       WHERE link = 'student_dashboard.php' AND title LIKE
    'My Profile%';
47  UPDATE menu_items SET link = 'dashboard.php'     WHERE link = 'student_dashboard.php' AND title =
    'Student Dashboard';
48  UPDATE menu_items SET link = 'notifications.php' WHERE link = 'student_dashboard.php#events';
49  UPDATE menu_items SET link = 'attendance.php'    WHERE link = 'student_dashboard.php#attendance';
50  UPDATE menu_items SET link = 'events_live.php', title = 'Event Finder (Fetch API)', min_role =
    'student', icon = 'bi-lightning-charge'
51      WHERE link = '../Practical-13/index.html';
52
53  -- 3. account status and a role check
54  ALTER TABLE users ADD COLUMN IF NOT EXISTS status ENUM('active', 'disabled') NOT NULL DEFAULT
    'active' AFTER role;
55  ALTER TABLE users ADD CONSTRAINT IF NOT EXISTS chk_users_role CHECK (role IN ('admin', 'editor',
    'student'));
```

The counters read `students` (status, `user_id`), `events` (`event_date`, `status`), `registrations` (`status`), `users` (`role`, `status`) and `audit_logs` (`status`, `created_at`). The relationships are unchanged from migration 001: one account is linked to at most one student record (`students.user_id`, `ON DELETE SET NULL`); students and events have many registrations (`ON DELETE CASCADE`); audit entries point at their account (`ON DELETE SET NULL`, so deleting an account keeps its entries and username). The menu rows of a new installation (`database/studenthub_demo.sql`) are:

id	title	link	min_role	Seen by
1	Student Dashboard	`dashboard.php`	student	all roles
2	My Profile	`profile.php`	student	all roles
3	Campus Events	`notifications.php`	student	all roles
5	Attendance	`attendance.php`	editor	editor, admin
6	Admin Management Console	`admin_dashboard.php`	admin	admin
7	Student Management (CRUD)	`admin_dashboard.php?tab=students`	admin	admin
8	Campus Event Manager	`admin_dashboard.php?tab=events`	admin	admin
9	Event Finder (Fetch API)	`events_live.php`	student	all roles
10	Security Audit Logs	`admin_dashboard.php?tab=audits`	admin	admin

Rows 1, 2, 3, 5 and 9 point at pages that the fixed sidebar groups already contain, so only an administrator gets a *Shortcuts* group (rows 6, 7, 8 and 10). The demo data has no `faculty.php` row (id 4); the live database keeps it and the portal does not show it.

4.1 Running the application

Create `studenthub_db` from `database/studenthub_demo.sql` (it already contains migrations 001–005; an existing database receives the missing migrations in order after a `mysqldump` backup), create the first administrator with `php tools\create_demo_user.php --db=studenthub_db --username=admin01 --email=admin01@example.com --role=admin` (the password is prompted; `--role=editor` / `--role=student` for further accounts) and run `php -S localhost:8000 -t studenthub`. An administrator lands on the Admin Console after signing in; students and editors land on the dashboard.

5. Implementation

5.1 Live statistics and latest data

`dashboard_stats()` counts everything with one aggregate query per table group; conditional `SUM()` expressions give the breakdowns, and `COALESCE` turns the sum of an empty table into 0. The Admin Console calls it on every page load, so the cards always show the current state of the database. Each card links to the matching tab.

studenthub/includes/dashboard.php (lines 9–22) – the five counters

```
 9  function dashboard_stats(mysqli $db): array
10  {
11      $one = fn (string $sql): array => $db->query($sql)->fetch_assoc();
12      $students = $one("SELECT COUNT(*) AS total, COALESCE(SUM(TRIM(status) = 'Inactive'), 0) AS
    inactive, COALESCE(SUM(user_id IS NOT NULL), 0) AS linked FROM students");
13      $events = $one("SELECT COUNT(*) AS total, COALESCE(SUM(event_date >= CURDATE()), 0) AS upcoming,
14                             COALESCE(SUM(event_date >= CURDATE() AND status = 'open'), 0) AS
    open_upcoming FROM events");
15      $registrations = $one("SELECT COUNT(*) AS total, COALESCE(SUM(status = 'registered'), 0) AS
    registered, COALESCE(SUM(status = 'attended'), 0) AS attended,
16                                    COALESCE(SUM(status = 'cancelled'), 0) AS cancelled FROM
    registrations");
17      $users = $one("SELECT COUNT(*) AS total, COALESCE(SUM(role = 'admin'), 0) AS admins,
    COALESCE(SUM(role = 'editor'), 0) AS editors,
18                             COALESCE(SUM(role = 'student'), 0) AS students, COALESCE(SUM(status =
    'disabled'), 0) AS disabled FROM users");
19      $audit = $one("SELECT COUNT(*) AS total, COALESCE(SUM(status = 'SECURITY_BLOCK' AND created_at >=
    NOW() - INTERVAL 7 DAY), 0) AS blocked_week FROM audit_logs");
20      $int = fn (array $row): array => array_map('intval', $row);
21      return ['students' => $int($students) + ['active' => (int) $students['total'] - (int)
    $students['inactive']], 'events' => $int($events),
22              'registrations' => $int($registrations), 'users' => $int($users), 'audit' =>
    $int($audit)];
```

`latest_registrations()` joins `registrations`, `students` and `events` (newest first, `LIMIT ?` bound as an integer), `newest_accounts()` lists the five latest accounts with role and status, and the *Recent system activity* table is `audit_list()` without filters, limited to six rows. The Overview tab also shows the applied migrations (`schema_migrations`) and how many student records are linked to an account.

5.2 Role-based navigation

Every `menu_items` row has a minimum role. `menu_for_role()` compares the rank of the user's role with the rank of the row and keeps the row only if `portal_page_exists()` confirms that the link is a page of this portal (a plain `name.php`, optionally with `?query` or `#part`, that exists as a file). Links such as `faculty.php` or `../Practical-13/index.html` can therefore never appear.

studenthub/includes/navigation.php (lines 7–26) – role hierarchy and menu filter

```
 7  const ROLE_RANK = ['student' => 1, 'editor' => 2, 'admin' => 3];
 8
 9  // True if the link points to an existing page of this portal ("page.php", "page.php?tab=x",
    "page.php#part")
10  function portal_page_exists(string $link): bool
11  {
12      $page = preg_split('/[?#]/', $link, 2)[0];
```

```
13      return preg_match('/^[A-Za-z0-9_\-]+\.php$/', $page) === 1 && is_file(dirname(__DIR__) .
    DIRECTORY_SEPARATOR . $page);
14  }
15
16  // The menu rows this role may see, in table order: [['title', 'link', 'icon', 'min_role'], ...]
17  function menu_for_role(mysqli $db, string $role): array
18  {
19      $rank = ROLE_RANK[$role] ?? 0;
20      $items = [];
21      foreach ($db->query('SELECT title, link, icon, min_role FROM menu_items ORDER BY id')-
    >fetch_all(MYSQLI_ASSOC) as $item) {
22          if ($rank >= (ROLE_RANK[$item['min_role']] ?? PHP_INT_MAX) &&
    portal_page_exists($item['link'])) {
23              $items[] = $item;
24          }
25      }
26      return $items;
```

`sidebar.php` (lines 32–47) calls `menu_for_role()` for the signed-in role, keeps the rows whose link is not already in its fixed groups and inserts them as a *Shortcuts* group after *Campus*; every value is printed with the escaping helper `e()` (`htmlspecialchars` with `ENT_QUOTES`). The console's navigation box lists the same rows. The links are only a convenience: every page behind them checks the permission again on the server.

5.3 One consistent access check and 403 page

Permissions are listed explicitly per role. The Admin Console starts with `require_permission("admin.console")`, and each action asks for its own permission again (`students.manage`, `events.manage`, `users.manage`, `inquiries.manage`). An editor holds only `portal.view`, so an editor is treated exactly like a student here.

studenthub/includes/auth.php (lines 20–28) – permissions per role

```
20  const STUDENTHUB_PERMISSIONS = [
21      'portal.view'     => ['admin', 'editor', 'student'],   // dashboard, profile, academic and campus
    pages
22      'admin.console'   => ['admin'],                        // Admin Console (admin_dashboard.php)
23      'students.manage' => ['admin'],
24      'events.manage'   => ['admin'],
25      'users.manage'    => ['admin'],
26      'audit.view'      => ['admin'],
27      'inquiries.manage' => ['admin'],                       // stored Help & Contact inquiries
    (Practical 7)
28  ];
```

studenthub/includes/auth.php (lines 272–282) – audit entry and the shared 403 page

```
272  function require_permission(string $permission): array
273  {
274      $user = require_login();
275      if (!role_can($user['role'], $permission)) {
276          global $conn;
277          log_audit($conn, $user['id'], $user['username'], 'ACCESS_DENIED',
278              "Role '{$user['role']}' lacks permission '$permission' for " .
    basename($_SERVER['SCRIPT_NAME'] ?? ''), 'SECURITY_BLOCK');
279          include dirname(__DIR__) . '/403.php';
280          exit;
281      }
282      return $user;
```

`require_permission()` first runs `require_login()` (signed in, not idle longer than the timeout, account re-read from the database). A missing permission writes `ACCESS_DENIED` with the status `SECURITY_BLOCK` and the reason (“Role 'editor' lacks permission 'admin.console' for admin_dashboard.php”), then includes `403.php`, which sends HTTP 403 and shows the signed-in user, the role and the notice that the attempt was recorded.

5.4 Account status: disable and enable

The User Accounts tab shows every account with role, status, last login and registration time. For every account except the administrator's own (marked *Current session*) it offers Disable / Enable and Delete as POST forms with the CSRF token; the console checks the token for every POST before any action runs (`require_valid_csrf()`).

studenthub/includes/dashboard.php (lines 61–81) – rules for disabling an account

```
 61  function set_account_status(mysqli $db, int $id, string $status, int $current_user_id): ?string
 62  {
 63      $account = find_account($db, $id);
 64      if ($account === null) {
 65          return 'That account was not found.';
 66      }
 67      if ($status === 'disabled') {
 68          if ($id === $current_user_id) {
 69              return 'You cannot disable your own account.';
 70          }
 71          if ($account['role'] === 'admin' && $account['status'] === 'active' &&
    active_admin_count($db) <= 1) {
 72              return 'The last active administrator cannot be disabled.';
 73          }
 74      }
 75      if ($account['status'] === $status) {
 76          return "The account “{$account['username']}” is already $status.";
 77      }
 78      $stmt = $db->prepare('UPDATE users SET status = ? WHERE id = ?');
 79      $stmt->bind_param('si', $status, $id);
 80      $stmt->execute();
 81      return null;
```

The POST handler in `admin_dashboard.php` (lines 180–212) requires `users.manage`, calls `set_account_status()`, writes `USER_DISABLED` or `USER_ENABLED` with the account's id, name and role, and redirects to the User Accounts tab with a message (“Account “neha_editor” was disabled. It can no longer sign in and its open sessions end.”). Deleting applies the same protections and writes `DELETE_USER`.

The status takes effect everywhere because the portal re-reads the account on every request. `account_block_reason()` returns a reason for a disabled account (any role) or for a student account whose linked record is inactive (Practical 11):

studenthub/includes/auth.php (lines 195–200) – why an account may not sign in or keep a session

```
195  function account_block_reason(array $row): ?string
196  {
197      if (($row['account_status'] ?? 'active') === 'disabled') {
198          return 'the account is disabled';
199      }
200      return account_is_deactivated((string) $row['role'], $row['student_status'] ?? null) ? 'the
     linked student record is inactive' : null;
```

- **Open session:** `validate_session()` calls `refresh_user_from_database()`; for a blocked account it writes `SESSION_REVOKED` (“Session ended: the account is disabled.”), deletes the session data, issues a new empty session ID and redirects to `login.php?status=access_disabled`.
- **New sign-in:** `login.php` checks the status only **after** `password_verify()` succeeded. A disabled account gets HTTP 403, no session, a `LOGIN_DENIED` entry and the generic message *“Your access to StudentHub is currently disabled. Please contact the administrator.”*; a wrong password still gets the normal *“Invalid username/email or password.”*, so the status is never revealed to someone without the password.
- **Protection:** an administrator cannot disable or delete the own account; the last active administrator cannot be disabled or deleted (a defence in depth – see Section 10).

5.5 Audit log and filters

`audit_filters()` reads its own query parameters (`audit_action`, `audit_status`, `audit_user`). Action and status are accepted only if they are in the list of existing actions or in `AUDIT_STATUSES`; anything else is ignored. The user filter is at most 50 characters and is bound into a `LIKE` with `%` and _ escaped, so SQL or wildcards in it are plain text. `audit_list()` builds the `WHERE` clause only from these three values, binds them as parameters and returns the newest 100 matching entries (`ORDER BY id DESC LIMIT ?`).

studenthub/includes/dashboard.php (lines 91–100) – whitelisted audit filters

```
 91  function audit_filters(array $query, array $actions): array
 92  {
 93      $text = fn (string $key): string => is_string($query[$key] ?? null) &&
     mb_check_encoding($query[$key], 'UTF-8') ? trim($query[$key]) : '';
 94      $action = $text('audit_action');
 95      $status = $text('audit_status');
 96      return [
 97          'action' => in_array($action, $actions, true) ? $action : '',
 98          'status' => in_array($status, AUDIT_STATUSES, true) ? $status : '',
 99          'user'   => mb_substr($text('audit_user'), 0, 50),
100      ];
```

Security-relevant entries written by the portal code (status in brackets):

Action	Written when	Status	Source
USER_LOGIN	successful sign-in	SUCCESS	login.php
LOGIN_FAILED	wrong password or unknown account	FAILED	login.php
LOGIN_DENIED	correct password, but the account is disabled or the student record inactive	SECURITY_BLOCK	login.php
SESSION_REVOKED	open session of a blocked account ends	SECURITY_BLOCK	includes/auth.php
SESSION_TIMEOUT	session ended after inactivity	INFO	includes/auth.php
ACCESS_DENIED	missing permission (HTTP 403)	SECURITY_BLOCK	includes/auth.php
CSRF_REJECTED	POST without a valid token	SECURITY_BLOCK	includes/auth.php, logout.php, api/events.php
USER_DISABLED / USER_ENABLED	account status changed	SUCCESS	admin_dashboard.php
DELETE_USER	account deleted (username and role in the details)	SUCCESS	admin_dashboard.php
USER_LOGOUT	sign-out by POST	SUCCESS	logout.php

Other entries record student, event, inquiry and export actions of the other practicals (for example `CREATE_STUDENT`, `UPLOAD_REJECTED`, `EVENT_REGISTER`, `EXPORT_CSV`). The portal has no function to edit or delete audit entries; deleting an account only clears the link (`user_id` becomes NULL) and keeps the username.

6. Validation and Security

- **Server-side authorisation:** the console, every tab and every POST action check a permission on the server; hiding a link is never the only protection. **CSRF:** account actions are POST forms with the session token; a GET request with `action=disable_user` and a POST without a token change nothing (tested).
- **Prepared statements and escaping:** every query with user input binds its parameters and filter values are whitelisted or bound as text; names, e-mail addresses, menu values and audit details are printed with `htmlspecialchars()` / `e()`, the statistics are cast to integers.

- **Sessions and database rules:** a disabled account loses its session at the next request and sees only one generic message after a correct password; `chk_users_role` refuses an invalid role and the `ENUM` of `users.status` refuses (or does not store) an invalid status – both tested in the migration suite.
- **Audit trail:** denied access, refused sign-ins, revoked sessions, CSRF rejections and account changes are recorded with user, IP address, user agent and status.

7. Output Screenshots

Taken by `tests/browser_dashboard.py` in Microsoft Edge (1280 px wide, full page) on a disposable database with fictional accounts (`portal_admin`, `neha_editor`, `riya_shah`, `kunal_desai`). The screenshots were made when migration 004 was the latest one, so the console does not yet show the *Inquiries* tab added later with migration 005.

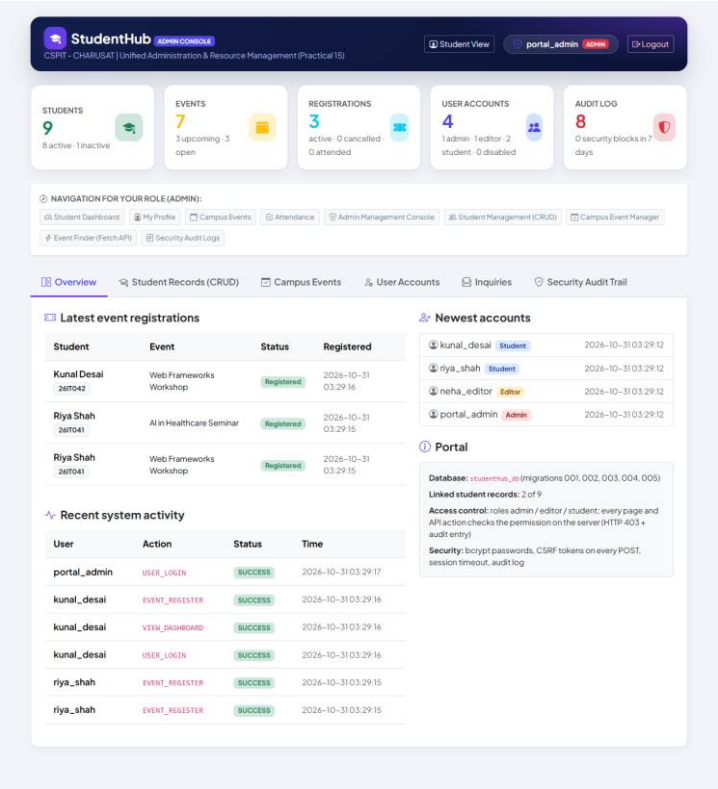


Figure 1: Admin Console overview: five live counters, navigation for the admin role, latest event registrations, newest accounts and recent activity (01-admin-dashboard.png)

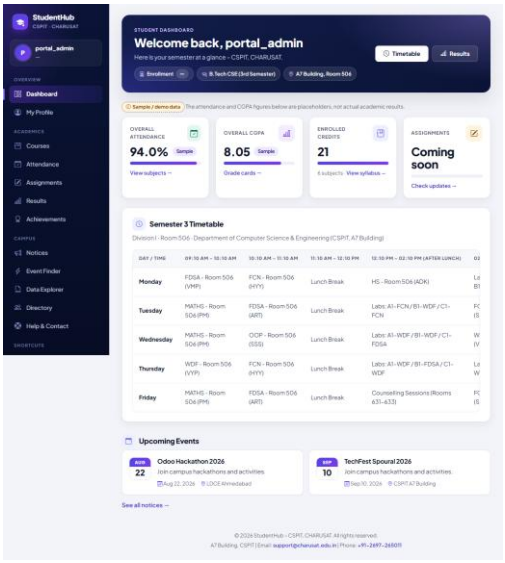


Figure 2: Administrator's sidebar with the Shortcuts group taken from menu_items (Admin Management Console) (02-admin-navigation.png)

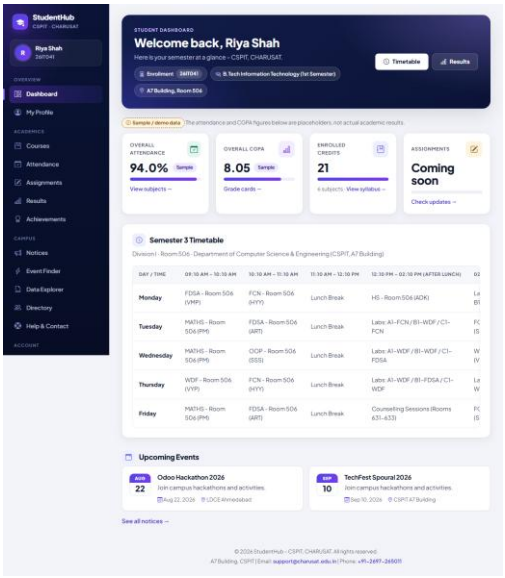


Figure 3: Student's sidebar: the same fixed groups, no Shortcuts and no administrator link (03-student-navigation.png)

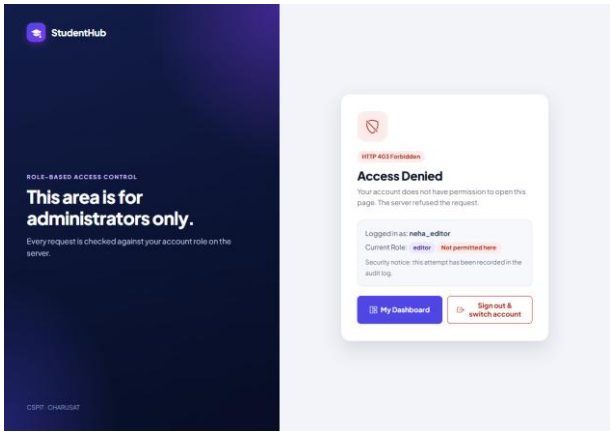


Figure 4: Editor opening the Admin Console: the shared HTTP 403 "Access Denied" page with user and role (04-editor-access-denied.png)

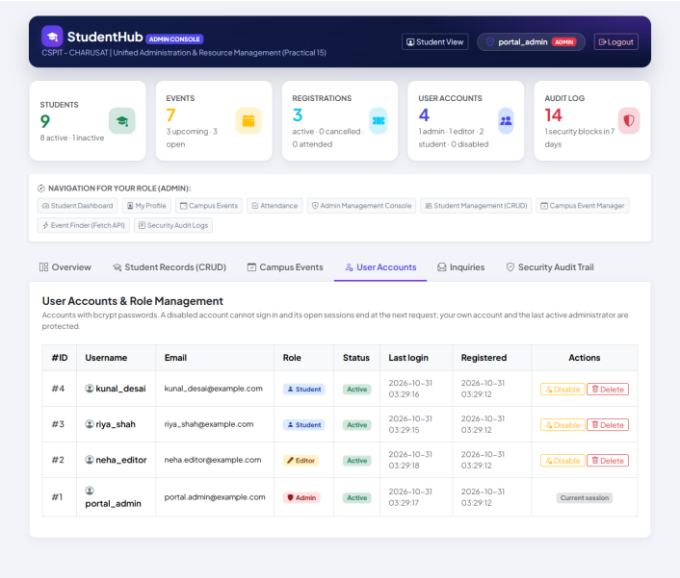


Figure 5: User Accounts tab: role and status badges; Disable and Delete for other accounts, "Current session" for the own one (05-user-accounts.png)

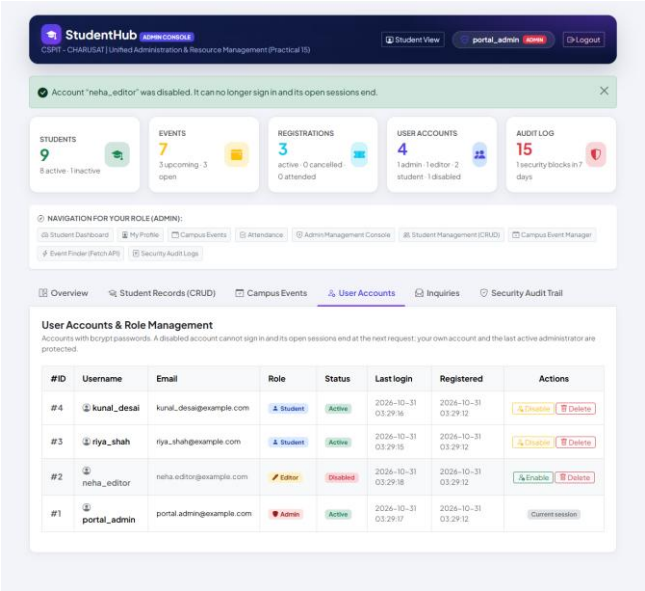


Figure 6: The editor account disabled: confirmation message, status Disabled, counter "1 disabled", Enable button (06-account-disabled.png)

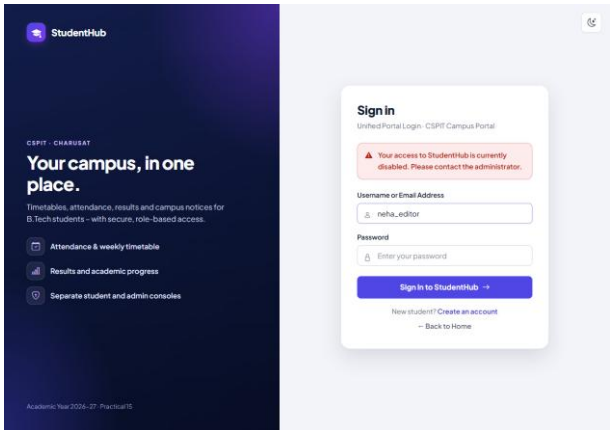


Figure 7: The disabled editor signing in with the correct password: generic refusal message (07-disabled-login-denied.png)

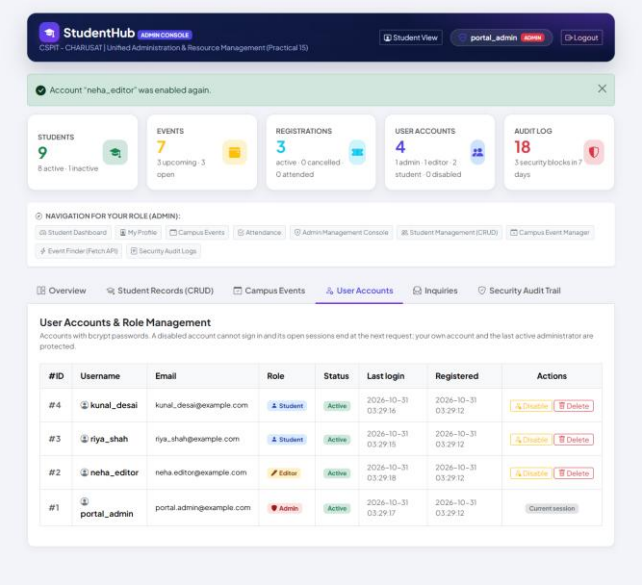


Figure 8: The account enabled again: status Active, counter back to "0 disabled" (08-account-enabled.png)

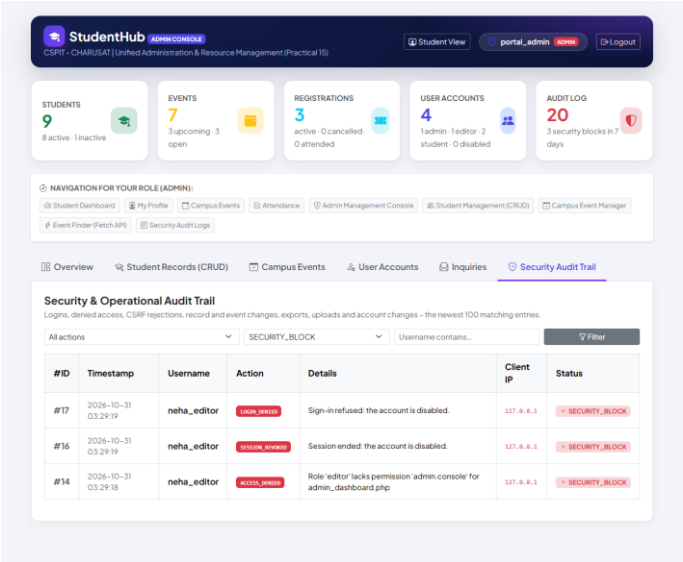


Figure 9: Audit log filtered to SECURITY_BLOCK: LOGIN_DENIED (#17), SESSION_REVOKED (#16) and ACCESS_DENIED (#14) with their details (09-audit-log-security.png)

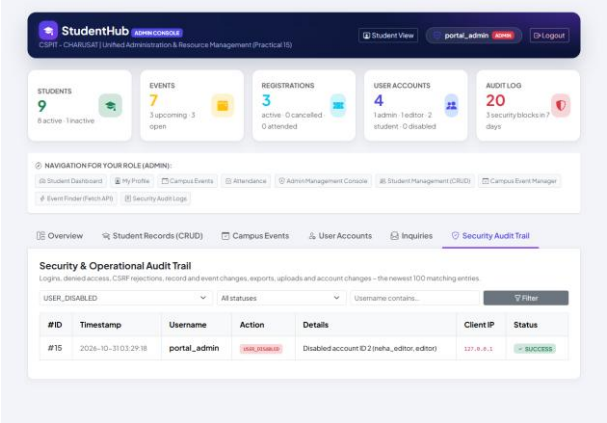


Figure 10: Audit log filtered to the action USER_DISABLED: the account change with the administrator's name (10-audit-log-account-change.png)

8. Test Cases and Results

Executed on 31 October 2026 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and Windows 11; browser tests in Microsoft Edge (headless, Playwright). Every write test used a disposable database (the HTTP suite builds it from `database/studenthub_demo.sql`); the live `studenthub_db` received migration 004 only after a fresh verified backup, and its accounts (including password hashes), students, events, registrations and categories were verified unchanged.

Suite	Result	Output file
`tests/test_dashboard.py` – HTTP tests	**36 / 36 passed**	`tests/test_dashboard_output.txt`
`tests/browser_dashboard.py` – Microsoft Edge + screenshots	**15 / 15 passed**	`tests/browser_dashboard_output.txt`
`database/tests/test_004_admin_dashboard.py` – fixture / real dump	**18 / 18** and **18 / 18 passed**	`database/tests/test_004_output.txt`
`test_004_admin_dashboard.py --verify-live` – live database (read-only)	**5 / 5 passed**	`database/tests/test_004_verify_live_output.txt`
Live checks of migrations 001–003 after 004	24 / 24, 4 / 4, 5 / 5	`database/tests/*_verify_live_output.txt`

tests/test_dashboard.py – 36 checks by group:

Group	Checks	What is verified (expected result = actual result in every case)
Live statistics	5	Five counters equal the database; accounts split by role with disabled ones counted; newest 3 registrations (student + event, newest first); five newest accounts; recent activity (up to 6)
Role-based navigation	6	Student and editor sidebars have no admin links; admin gets Shortcuts from `menu_items`; never `faculty.php` or `../Practical-13`; HTML in a menu title or icon escaped; the console's navigation box lists exactly the admin's rows
Access-denied page	4	Student and editor: every console tab gives the same HTTP 403 page, each attempt logged; a POST to disable an account gives 403 and changes nothing
Account status	13	POST forms with tokens (none for the own account); GET and token-less POST change nothing; disable → 303, `USER_DISABLED`; open session ends (`SESSION_REVOKED`); sign-in refused (403, `LOGIN_DENIED`); wrong password gets the normal message; disabling twice; own account protected; disabled student refused; enable → `USER_ENABLED` and sign-in works; unknown id; deleting another account is logged with the username
Audit filters	6	Action, status, user and combined filters return exactly the matching entries (newest first); unknown values ignored, SQL in the user filter is text; no PHP warnings in the server log
Clean-up	2	Disposable database removed; live `studenthub_db` not touched

tests/browser_dashboard.py – 15 checks in Microsoft Edge:

#	Browser check	Result
1	Three registrations made by two students on the Campus Events page	PASS
2	Dashboard: live counters equal the database	PASS
3	Latest registrations show the students and events just registered	PASS
4–6	Admin sidebar has Shortcuts to the console tabs; student and editor sidebars have no admin links	PASS
7	Editor opening the Admin Console → HTTP 403 "Access Denied" (same page as for students)	PASS
8–11	Admin disables the editor; the editor's open session ends; sign-in refused; after Enable the editor signs in	PASS
12–13	Audit log filtered to SECURITY_BLOCK and to the action USER_DISABLED shows the expected entries	PASS
14–15	No JavaScript errors (only the intended 403 responses); no PHP warnings in the server log	PASS

The browser run also reported the known 404 for `/favicon.ico` (the portal has no favicon) as information.

Migration 004 (18 checks): only the intended structural changes; menu rows converted and every other row, account (incl. password hashes), student, event, registration, category and audit entry unchanged; invalid role and status refused; idempotent; rollback, re-apply and the precondition stop work; a fresh install equals an upgraded database.

Regression (TEST_REPORT §11): `test_api` 72 / 72, `test_events` 94 / 94, `test_students` 170 / 170, `test_auth` 132 / 132, migrations 001–003 88 / 88, 22 / 22, 27 / 27 and five Edge suites (75 checks) – all passed. In the final complete run of Practical 15 (`tests/run_all_output.txt`) the Practical 14 suites passed again (36, 18 and 15, none failed).

9. Problems Encountered and Fixes

Problem	Fix
`menu_items.min_role` used the value `user`, which is not a portal role, and several rows linked to standalone pages (`../Practical-13/index.html`) or to anchors that do not exist	Migration 004 converts `user` to `student`, restricts the column to the three roles and points those links at the integrated pages
The live database contains a menu link to the non-existent `faculty.php`	The row is kept (no data is deleted); `portal_page_exists()` hides every link to a missing page
The sidebar printed its link values without escaping – harmless while they were fixed in the code, but now they come from `menu_items`	Every title, link and icon is printed with `e()`; a test inserts HTML into a menu row and checks that it is escaped
Disabling an account must also end the sessions that are already open	The account is re-read on every request; `account_block_reason()` now also covers `users.status`, so the session ends at the next request (SESSION_REVOKED)
The refusal must not reveal the account status to someone without the password	The status is checked only after `password_verify()`; a wrong password always gets the normal message

10. Known Limitations

- There is no form to edit `menu_items`; rows are changed in the database or by migrations.

- The “last active administrator” protection is a defence in depth: it cannot be reached through the interface, because an administrator cannot disable or delete the own account.
- The Admin Console loads Bootstrap from a CDN; the portal has no favicon (one 404 per browser session).
- `audit_logs` is an ordinary table: anyone with direct database access could change it. The portal itself offers no way to edit or delete entries.
- There is no limit on repeated sign-in attempts (a project-wide limitation, see `TEST_REPORT.md`).

11. Conclusion

Practical 14 is now part of the one StudentHub portal instead of a separate dashboard on another database. The Admin Console counts students, events, registrations, accounts and audit entries live from `studenthub_db`, lists the latest registrations, accounts and activity, and builds the navigation from `menu_items` according to the role hierarchy student < editor < admin. Every request by a signed-in student or editor to the console ends on the same HTTP 403 page with an audit entry, administrators can disable and re-enable accounts with immediate effect on open sessions and sign-ins, and the audit log can be filtered by action, status and user. The behaviour is verified by 36 HTTP checks, 15 Microsoft Edge checks and 18 + 18 + 5 migration checks, all passed, and documented with ten genuine screenshots.

12. References

- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php
- PHP Manual – htmlspecialchars: https://www.php.net/manual/en/function.htmlspecialchars.php
- OWASP – Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- OWASP – Logging Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Bootstrap 5.3 documentation: https://getbootstrap.com/docs/5.3/
- Playwright for Python: https://playwright.dev/python/
- Project files: `TEST_REPORT.md` (section 11), `database/README.md` (migration 004), `README.md` (Admin dashboard, Local Setup), `Practical-14/README.md` (standalone version)
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder `studenthub/`, commit `05927b3`)